



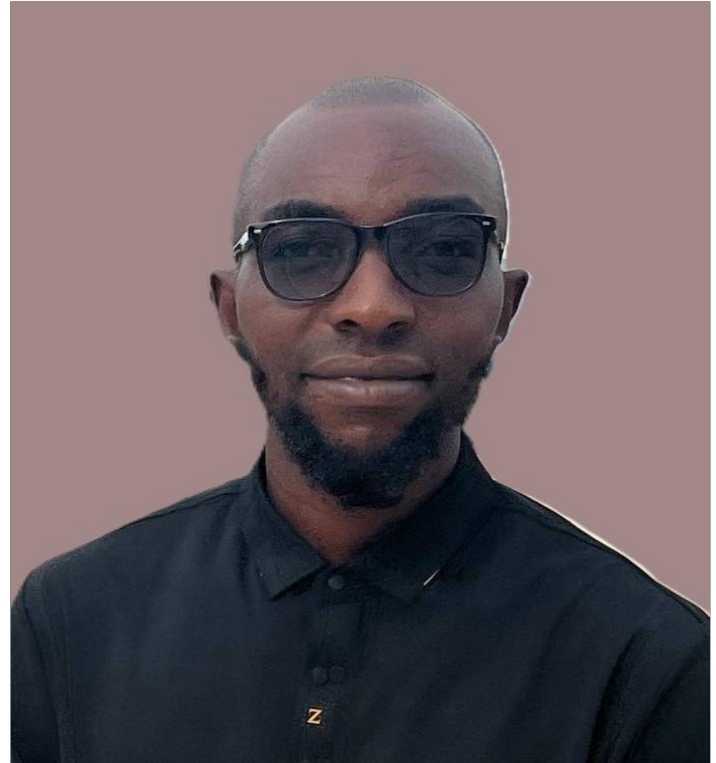
Lisk Summer Bootcamp

Etette Etok

Software Engineer & Web3 DevRel

Twitter: [@Em_Etok](#)

Linkedin: [Etette Etok](#)



Today's Agenda

- Vulnerabilities
- Best practices
- Other Development tools
- Scaffold Lisk



Vulnerabilities

Smart Contracts are at the heart of blockchain transactions and their integrity is essential to blockchain reliability and performance.

Solidity smart contracts are susceptible to various vulnerabilities that can lead to significant financial losses and reputational damage. Common vulnerabilities include reentrancy attacks, integer overflows and underflows, access control issues, denial-of-service (DoS) attacks and unchecked external calls.

Understanding these vulnerabilities and implementing robust security practices is crucial for developing secure and reliable smart contracts.

Why Smart Contracts Are Vulnerable

- **Immutability:** Once deployed most smart contracts cannot be changed, locking in any bugs.
- **Public Code:** Contracts are often open-source, allowing attackers to study them for weaknesses.
- **Financial Incentive:** Smart contracts often manage large sums of cryptocurrency, making them prime targets.
- **Complex Interactions:** Contracts interact with other contracts, users, and external data, increasing risk.

Common Smart Contract Vulnerabilities

- **Reentrancy Attacks:** Reentrancy occurs when an external contract is called before the original contract updates its state. This allows the attacker to re-enter the contract in a loop, potentially draining funds.
- **Logic Errors:** Smart contracts may behave contrary to their intended logic due to incorrect operators, boundary conditions or flawed algorithm implementation.
- **Access Control:** Critical functions may lack proper access restrictions, allowing unauthorized users to execute privileged operations.
- **Denial of Service:** Attackers can block contract functionality by causing transactions to run out of gas or always revert.
- **Uninitialized Storage Pointers:** Improperly initialized variables can point to unintended storage causing data corruption.
- **Frontrunning:** Attackers can observe pending transactions and submit their own with higher gas prices to execute first, potentially profiting from price movements or arbitrage opportunities.
- **Oracle Manipulation:** Reliance on centralized or manipulable price feeds can allow attackers to manipulate on-chain logic, especially with flash loans that can temporarily skew market prices.
- **Insecure Randomness:** Using predictable blockchain values like `block.timestamp`, `block.number`, or `blockhash` for randomness allows attackers to predict outcomes.

Best Practices

Simplicity: Simplicity in smart contract development means writing concise, focused code that avoids unnecessary complexity. The more complex a contract, the larger its attack surface (i.e., the number of potential entry points for exploits). Complex code increases the likelihood of bugs, makes auditing harder and raises gas costs. Keep code minimal to reduce attack surfaces.

Explicitness: Explicitness ensures that code is easy to understand and unambiguous, reducing the chance of misinterpretation by developers, auditors or attackers. Ambiguous logic such as unclear variable names or undocumented assumptions can lead to errors or exploitable vulnerabilities. Avoid ambiguous logic; use clear variable names and comments.

Modularity: Modularity involves splitting a smart contract into smaller, self-contained components that handle specific tasks. This approach reduces complexity, improves reusability and makes testing and auditing easier. Break contracts into smaller, reusable components.

Stay Defensive: Assumes that all external inputs (whether from users, other contracts or oracles) could be malicious. This principle encourages multiple layers of security to protect against attacks like reentrancy, unauthorized access, or malicious data. By anticipating worst-case scenarios, developers can build robust contracts that withstand adversarial conditions.

Continuous Learning: Stay updated with Solidity versions and evolving threats.

Smart Contract Development Tools

Remix: Remix is a browser-based Integrated Development Environment (IDE) designed for quick prototyping and debugging of Solidity smart contracts, making it an excellent choice for beginners and developers seeking rapid development due to its intuitive interface and built-in tools.

Hardhat: Hardhat is an advanced Ethereum development environment that provides robust features for testing, debugging and deploying smart contracts, tailored to professional developers working on complex projects who need scalability and flexibility.

Truffle: Truffle is a comprehensive framework that streamlines the process of building, testing and deploying smart contracts, offering a full suite of tools including migration scripts, testing libraries and network management for end-to-end development.

Brownie: Brownie is a Python-based framework that simplifies smart contract development for Python developers by leveraging familiar syntax and tools like pytest, making it ideal for those already comfortable with Python ecosystems.

Foundry: Foundry is a fast, modular and extensible toolkit for Ethereum application development, emphasizing speed and flexibility with features like on-the-fly compilation and gas optimization for efficient workflows.

Scaffold-Lisk: Scaffold-Lisk is a specialized framework for building decentralized applications on the Lisk blockchain, providing pre-built React components, smart contract templates and integration tools to accelerate dApp development.

Scaffold-Lisk

Scaffold-Lisk is a specialized framework aimed at simplifying the creation of decentralized applications on the Lisk blockchain. It's built to help developers, especially those familiar with web development, by providing a ready-to-use setup that reduces the complexity of starting a blockchain project. It's a fork of Scaffold-OP, tailored for Lisk and includes support for Superchain testnets.

Scaffold-Lisk offers several features that enhance developer productivity and ease of use. One notable feature is contract hot reloading, which allows the frontend to automatically adapt to changes in the smart contract as it is edited, eliminating the need for manual redeployments and speeding up the development cycle. Another significant aspect is the inclusion of custom hooks, a collection of React hooks that simplify interactions with smart contracts while providing TypeScript autocompletion for better code suggestions and error reduction.

It also supports burner wallet and local faucet functionalities, allowing developers to test their applications without needing real funds or complex testing environments.

The End

Questions?

ASSIGNMENT

- Identify and fix the vulnerabilities in this simple smart contract to make it secure. Add your custom attack function to attack the smart contract and call the withdraw function.
- Submit the repo link to your audited source code. Add the deployed contract explorer link in the [ReadMe.md](#) file

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract VulnerablePiggyBank {
```

```
    address public owner;
```

```
    constructor() { owner = msg.sender }
```

```
    function deposit() public payable {}
```

```
    function withdraw() public { payable(msg.sender).transfer(address(this).balance); }
```

```
    function attack() public { }
```

```
}
```